Desarrollo basado en especificaciones con GitHub Copilot

Gestor de incidencias en C# y Windows Forms

Santiago Nicolás Blacio Quimis

Frank Emilson Cárdenas Figueroa

Universidad Técnica de Machala

Programación Avanzada en C#

Paralelo 4A

Docente: Ing. Cartuche

25 de octubre de 2026

Introducción y alcance

El caso del primer parcial describe una empresa de software en Ecuador con pérdida de versiones, trabajo desorganizado, errores en producción, documentación insuficiente y falta de automatización. Es un escenario académico. No se dispone de mediciones de una empresa identificada, por lo que este trabajo no atribuye ahorros ni mejoras porcentuales a una organización real.

El objetivo general es aplicar un proceso de desarrollo en C# donde los requisitos guíen la generación de código, su comprobación y la colaboración. Los objetivos específicos son formular criterios observables, construir una aplicación mediante GitHub Copilot en Visual Studio y conservar evidencias del historial y de las verificaciones.

La indicación del docente establece las fases de requerimientos, especificación, plan técnico, implementación con IA, verificación y evolución (Ing. Cartuche, comunicación personal, 23 de octubre de 2026). Los estudiantes deben comprender y revisar el resultado generado. El documento de la asignatura constituye la guía del caso. El video compartido no se utilizó como fuente porque no se recuperó su contenido.

El producto actual es una aplicación local Windows Forms sobre .NET Framework 4.7.2. Registra y edita incidencias, permite buscarlas y controla sus estados. Guarda información en XML. Esta entrega corresponde al proyecto GestionIncidenciasCopilot creado por el estudiante, no al prototipo web preliminar descartado.

Control de versiones y colaboración

Git permite conservar cambios en commits, comparar versiones y mantener ramas de trabajo. Pro Git describe el uso de ramas temáticas y líneas de integración para organizar cambios (Chacon & Straub, s. f.). En el caso, los commits separan requisitos, especificación, plan, entidades, persistencia, controlador, formularios, pruebas y automatización.

Merge integra historias. Rebase reorganiza commits sobre otra base y debe utilizarse con cuidado en ramas compartidas. Cherry-pick permite aplicar un cambio concreto. Para deshacer una modificación publicada se propone revertir mediante un nuevo commit, conservando el historial que utiliza el compañero.

Git Flow organiza funcionalidades, preparación de versiones y correcciones mediante distintas ramas. Trunk-based favorece integraciones frecuentes sobre la rama principal. Para dos integrantes se propone main estable y ramas breves con revisión. La demostración actual agrupa la funcionalidad en feature/gestor-incidencias. Es una simplificación académica y no demuestra todavía integración diaria.

Un pull request reúne cambios para revisión y discusión antes de integrarlos (GitHub, s. f.-a). Se publicaron main y feature/gestor-incidencias en el repositorio privado indicado por el estudiante. El PR #1 permanece en borrador y no se fusionó. La revisión auténtica del compañero está pendiente. No se atribuyen commits ni aprobaciones ficticias al segundo integrante.

Spec-Driven Development y uso de IA

SDD utiliza la especificación como referencia para decidir qué construir y cómo aceptarlo. GitHub Spec Kit estructura el trabajo mediante especificación, planificación y tareas antes de la implementación (GitHub, s. f.-b). Este proyecto aplica ese enfoque con documentos versionados. Se consultó Spec Kit, pero no se afirma haber instalado ni ejecutado su CLI.

Los requerimientos describen necesidades y restricciones. La especificación fija el modelo, los contratos del controlador, las reglas de persistencia y el comportamiento de los formularios. El plan técnico divide el trabajo en tareas T01 a T08. Las pruebas comparan el resultado con esos contratos. Una evolución debe cambiar primero el criterio correspondiente y conservar el vínculo con sus modificaciones.

Copilot Agent puede trabajar sobre archivos y ejecutar tareas dentro de Visual Studio (Microsoft, s. f.-a). La implementación actual y sus pruebas se generaron mediante solicitudes en el chat real de Copilot del proyecto. Las solicitudes se dividieron en cambios pequeños, se revisaron los archivos y se guardaron antes de compilar. El historial del chat permanece en el entorno del estudiante.

Un prompt adecuado identifica el documento de referencia, la tarea, las restricciones y el resultado que debe comprobarse (GitHub, s. f.-c). Ejemplo orientativo: implementar el controlador conforme a spec.md, conservar ID y fecha de creación al editar, persistir antes de confirmar éxito y rechazar transiciones inválidas. Este ejemplo resume el método y no pretende ser una transcripción literal del chat.

Codex coordinó el trabajo, operó Visual Studio y preparó documentación, evidencias y entrega. Los archivos operativos y la incorporación del proyecto de pruebas a la solución no se presentan como código generado por Copilot. La autoría de IA no exime a los integrantes de comprender y explicar las reglas.

Especificación del caso práctico

La entidad Incidencia contiene un identificador GUID, título, descripción, prioridad, estado y fechas UTC de creación y actualización. El título es obligatorio, se recorta y admite hasta 120 caracteres. La descripción es opcional y admite hasta 2000. Las prioridades válidas son Baja, Media y Alta.

Una incidencia nueva comienza Pendiente. Las transiciones autorizadas son Pendiente a EnProceso, EnProceso a Resuelta y Resuelta a EnProceso. Cualquier otra transición debe rechazarse sin cambiar memoria ni almacenamiento. El usuario puede editar título, descripción y prioridad, conservando identificador y fecha de creación.

Criterio de aceptación: dada una incidencia Pendiente, cuando se solicita pasar directamente a Resuelta, el controlador informa el error y conserva su estado previo. Otro criterio exige que un fallo de guardado no aparezca como una operación exitosa ni altere la colección confirmada.

La búsqueda consulta título y descripción sin distinguir mayúsculas. Los filtros opcionales de prioridad y estado se combinan mediante una condición conjunta. Sin coincidencias, el listado queda vacío. Sin selección, las acciones de editar y detalle permanecen deshabilitadas.

El archivo XML ausente representa una colección vacía. Un archivo presente pero corrupto, con versión no admitida, IDs duplicados o fechas inválidas produce un error explícito y bloquea escrituras durante la sesión. El sistema no interpreta un archivo ilegible como vacío. No incluye autenticación, eliminación, acceso multiusuario ni una base de datos de producción.

Arquitectura y formularios

La solución separa Entidades, Controladores, Datos y Formularios. Incidencia y sus enumeraciones representan el dominio. ControladorIncidencias valida entradas, controla transiciones y coordina la persistencia. IRepositorioIncidencias define el contrato de almacenamiento y RepositorioXmlIncidencias lo implementa. Esta distribución coincide con los conceptos de las prácticas de clase proporcionadas como guía.

FrmPrincipal presenta el DataGridView de solo lectura, búsqueda y filtros. FrmIncidencia permite registrar y editar, y utiliza ErrorProvider para señalar entradas inválidas. FrmDetalle muestra identidad, contenido, fechas y las siguientes transiciones permitidas. Los tres tienen archivos Designer.cs editables. Las reglas no dependen de eventos particulares de los controles.

Program carga el repositorio antes de abrir el listado y muestra un error explícito si los datos no son válidos. El archivo de usuario se ubica en %LOCALAPPDATA%/GestionIncidenciasCopilot/incidencias.xml. Las fechas se serializan en UTC con sufijo Z. El controlador devuelve copias para impedir que la interfaz modifique el estado interno accidentalmente.

Para guardar, el repositorio escribe un temporal único en el mismo directorio y fuerza el vaciado de buffers. Utiliza File.Replace cuando ya existe el destino y Move para la primera escritura. No borra previamente el archivo válido. Esta estrategia mejora la protección frente a fallos, pero no aporta coordinación de múltiples procesos. La limpieza de temporales tras ciertos fallos queda como mejora técnica.

La prueba visual a una escala de pantalla del 150 % detectó problemas de altura. Copilot agregó adaptación al área disponible y desplazamiento. Se conservaron los ajustes de diseñador realizados por el estudiante. La ejecución Release permitió verificar las acciones inferiores y completar los recorridos.

Pruebas, TDD y automatización

TDD plantea escribir una prueba que falle, implementar lo necesario y refactorizar conservando el resultado. Las buenas prácticas de pruebas de .NET favorecen casos comprensibles que protejan comportamientos útiles frente a regresiones (Microsoft, s. f.-b). En este proyecto las pruebas se generaron después de la aplicación. No se conserva un ciclo rojo-verde-refactor previo y no corresponde afirmar que se aplicó TDD completo.

El proyecto Pruebas es un ejecutable de aserciones sin paquetes externos. No utiliza xUnit, MSTest ni un adaptador de dotnet test. Devuelve código cero cuando todos los grupos pasan y uno ante cualquier fallo. Cada grupo contiene varias aserciones. Ocho grupos aprobados no equivalen a ocho pruebas individuales ni a cobertura total.

Los grupos verifican alta y límites, edición e identidad UTC, transiciones, filtros conjuntos, protección mediante copias, fallo al guardar sin cambio de memoria, lectura/escritura XML y protección ante datos corruptos. MSBuild compiló aplicación y pruebas en Release. La salida registrada muestra los ocho grupos aprobados.

GitHub Actions permite repetir compilación y verificaciones a partir de eventos del repositorio (GitHub, s. f.-d). El workflow usa Windows, configura MSBuild, compila el proyecto de pruebas con su referencia a la aplicación, ejecuta las aserciones y conserva los binarios como artefacto. Se activa en main, ramas feature y pull requests hacia main.

La vista del PR en Visual Studio mostró todas las comprobaciones opcionales superadas. La captura incluida del PR corresponde al momento anterior, cuando la comprobación aún estaba ejecutándose. No se adjuntó un log remoto detallado ni se inventó un ID de ejecución. El pipeline produce un paquete para Windows. No despliega a un servidor ni demuestra una entrega productiva.

Verificación manual y trazabilidad

Se rechazó un título vacío y se registró DEMO SDD - Error al iniciar sesion con prioridad Alta. La aplicación permitió pasar de Pendiente a EnProceso y luego a Resuelta. La edición conservó el identificador 81d5deab-a3f9-49d7-a49f-25317f9b9336 y la fecha de creación. Se comprobó la búsqueda por descripción sin distinguir mayúsculas y la combinación de filtros con y sin coincidencias.

Tras cerrar y reabrir la aplicación en Release, se conservaron identificador, prioridad, estado y fechas. Las capturas adjuntas documentan listado, detalle y persistencia después del reinicio. Una instancia anterior bloqueó el ejecutable Debug. Se completó la comprobación con Release y no se presenta Debug como verificado.

La secuencia del repositorio conserva requisitos (7cc7511), especificación (1554f64), plan (ea58db6), entidades (b03e4c3), XML (ba59b9c), controlador (b995abb), formularios (c848b66 y aefe422), listado y arranque (2ea2668), pruebas (ff93e93) y CI (00f1a4c). Los ajustes posteriores preservaron el trabajo del estudiante y actualizaron evidencias.

El PR de revisión se encuentra en https://github.com/FrankEmilson41-glitch/GestionIncidenciasCopilo/pull/1. El repositorio es privado y requiere acceso autorizado. El paquete de entrega incluye una copia del código y evidencia local para revisar el caso sin depender de una sesión autenticada.

La evidencia distingue lo realizado de lo propuesto: existen aplicación ejecutada, historial, ramas publicadas y PR en borrador. Falta la revisión auténtica del compañero y completar la organización mediante Issues, Project y milestone. Estas actividades deben acreditarse antes de afirmar cumplimiento total del bloque de gestión.

Gestión, propuesta de adopción y análisis crítico

GitHub Projects permite organizar issues y pull requests en vistas de tabla o tablero (GitHub, s. f.-e). Para este caso se propone un Kanban con Pendiente, En curso, En revisión y Terminado. Cada issue debe incluir requisito, criterio de aceptación y responsable. Un milestone del primer parcial agruparía las tareas de entrega. El tablero y el milestone se describen como propuesta, no como recursos ya creados.

El plan de dos semanas asigna la primera a investigación, especificación y construcción por tareas, y la segunda a pruebas, revisión y preparación de la exposición. Es una planificación sugerida y no un registro de horas ejecutadas. Santiago puede exponer el problema, Git y SDD. Frank puede explicar arquitectura, Copilot y verificación. Ambos deben poder justificar el comportamiento completo.

La principal ventaja de SDD es hacer explícito el criterio de aceptación. Su límite es que una especificación equivocada también puede producir código y pruebas coherentes entre sí, pero insuficientes para el usuario. Por ello, la revisión debe comenzar por la necesidad y los ejemplos negativos, no únicamente por comprobar una salida verde.

La persistencia XML simplifica la demostración local, pero limita concurrencia y atribución de cambios. Antes de un uso empresarial se requieren almacenamiento transaccional, autenticación, copias de seguridad y una estrategia de despliegue. Estas funciones ampliarían el alcance y deben especificarse antes de solicitar su generación a la IA.

Como indicadores para un piloto se proponen proporción de cambios vinculados a requisitos, revisiones del compañero y fallos detectados antes de integrar. No se presentan resultados cuantitativos sin medición. La siguiente evolución puede usar TDD real conservando primero una prueba fallida, después su implementación con Copilot y finalmente la refactorización.

Reflexiones y conclusiones

¿Qué problema resuelve Git? Conserva un historial recuperable y facilita la colaboración mediante ramas. En el caso permite identificar qué cambio incorporó una regla o introdujo un defecto, evitando depender de carpetas duplicadas con nombres de versión.

¿Por qué es importante DevOps? Hace reproducibles las verificaciones y reduce la dependencia de pasos manuales recordados por una persona. Su valor exige que las pruebas representen necesidades útiles y que exista una decisión de integración y entrega.

¿Cómo cambia la IA el rol del programador? Desplaza parte del esfuerzo hacia la especificación, la revisión y la comprobación. La persona debe detectar omisiones, comprender el código y justificar por qué el resultado satisface una necesidad. Una respuesta convincente de Copilot no demuestra corrección.

¿Qué práctica se adoptaría primero en la empresa? Se propone empezar por Git con un repositorio compartido, requisitos breves y revisión de cambios. Sobre esa base, automatizar compilación y pruebas evita integrar errores conocidos. La adopción debe acompañarse de una responsabilidad clara por cada criterio.

El caso demuestra un flujo de especificación, generación con Copilot, compilación y verificación en una aplicación Windows Forms funcional. La separación por capas permite explicar las reglas sin depender de la interfaz. El historial y el PR conservan el contexto de los cambios. La revisión del compañero y la gestión remota pendiente delimitan el resultado actual y constituyen los siguientes pasos para cerrar la entrega académica.

Referencias

Chacon, S., & Straub, B. (s. f.). Branching workflows. Pro Git. https://git-scm.com/book/en/v2/Git-Branching-Branching-Workflows

GitHub. (s. f.-a). Pull requests. GitHub Docs. https://docs.github.com/en/pull-requests/reference/pull-requests

GitHub. (s. f.-b). GitHub Spec Kit [Repositorio de software]. https://github.com/github/spec-kit

GitHub. (s. f.-c). Prompt engineering for GitHub Copilot. GitHub Docs. https://docs.github.com/en/copilot/concepts/prompting-and-context/prompt-engineering

GitHub. (s. f.-d). Building and testing .NET. GitHub Docs. https://docs.github.com/en/actions/tutorials/build-and-test-code/net

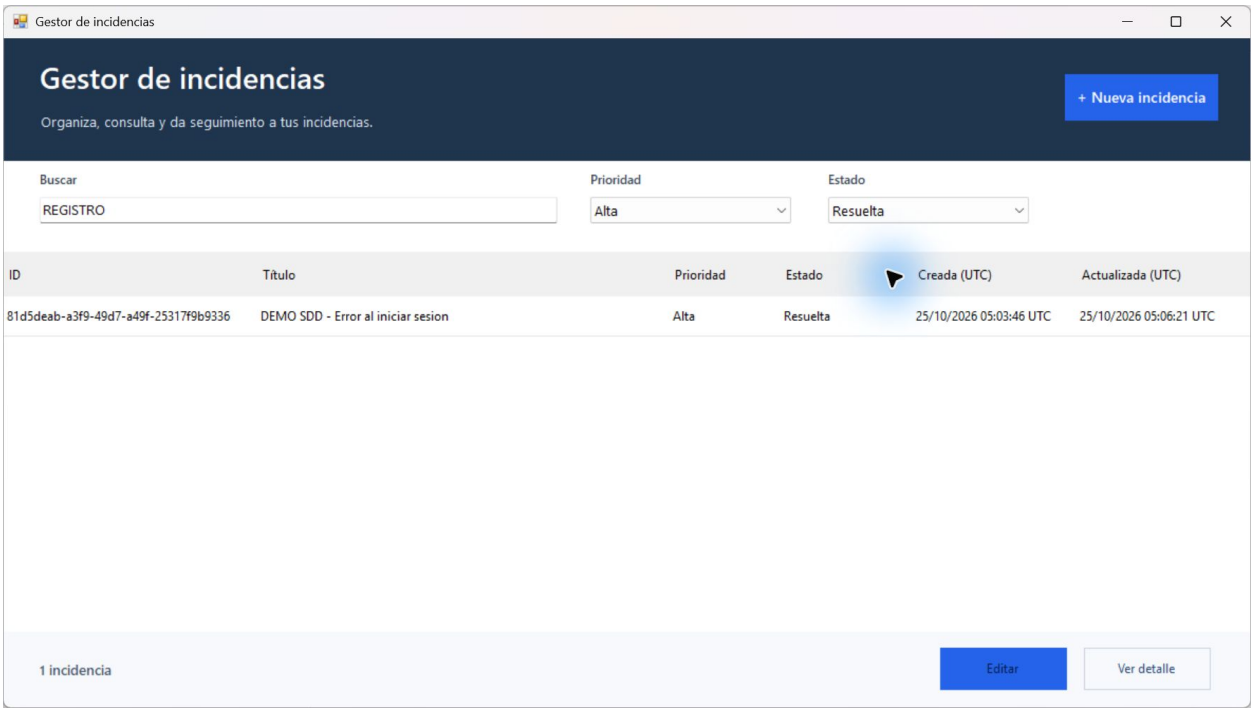
GitHub. (s. f.-e). About Projects. GitHub Docs. https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects

Microsoft. (s. f.-a). Use Copilot agent mode in Visual Studio. Microsoft Learn. https://learn.microsoft.com/en-us/visualstudio/ide/copilot-agent-mode

Microsoft. (s. f.-b). Unit testing best practices. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-best-practices

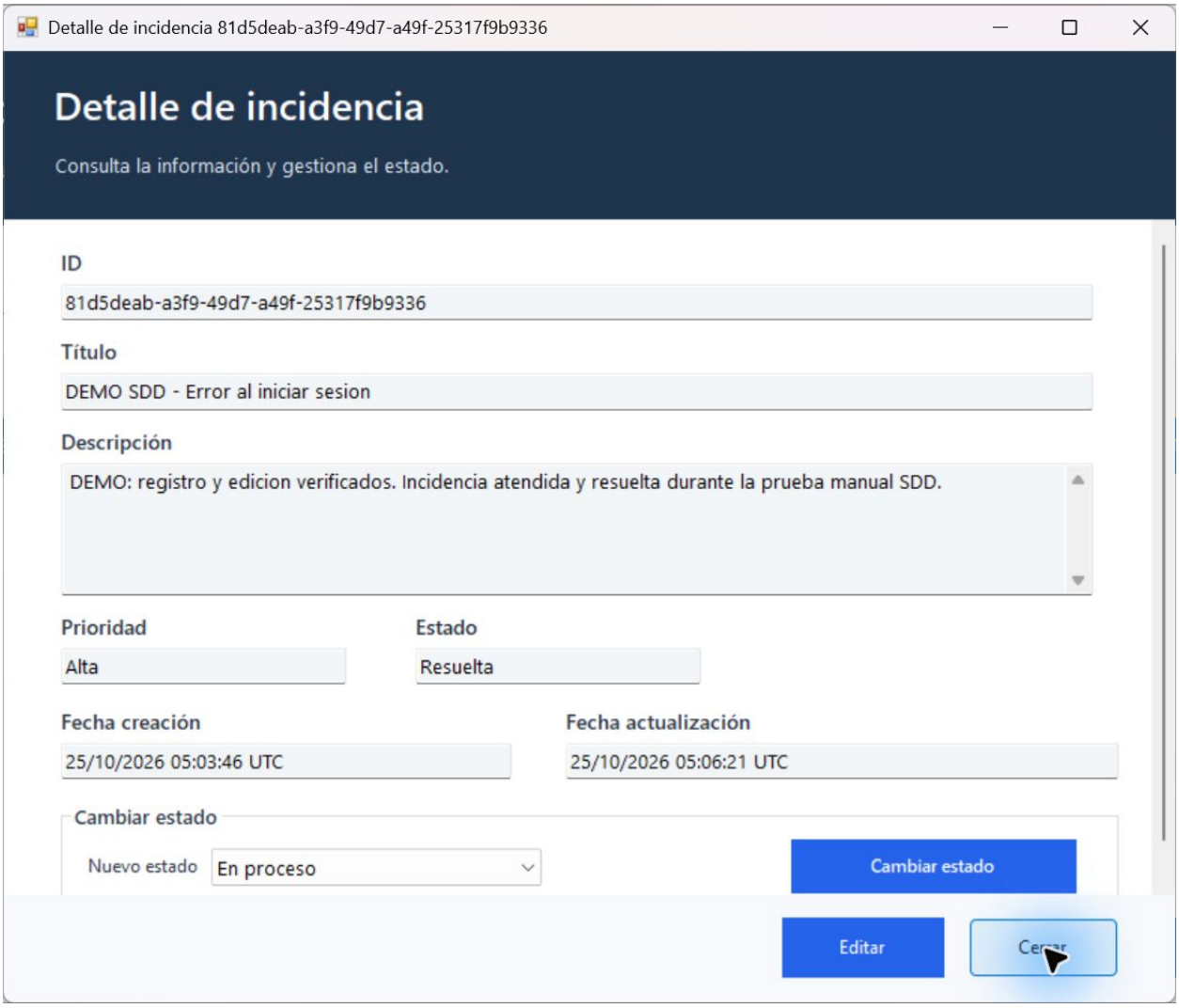
Proyecto Parcial I - PA. (s. f.). [Guía de actividad académica proporcionada por el estudiante].

Anexo A. Listado y filtros



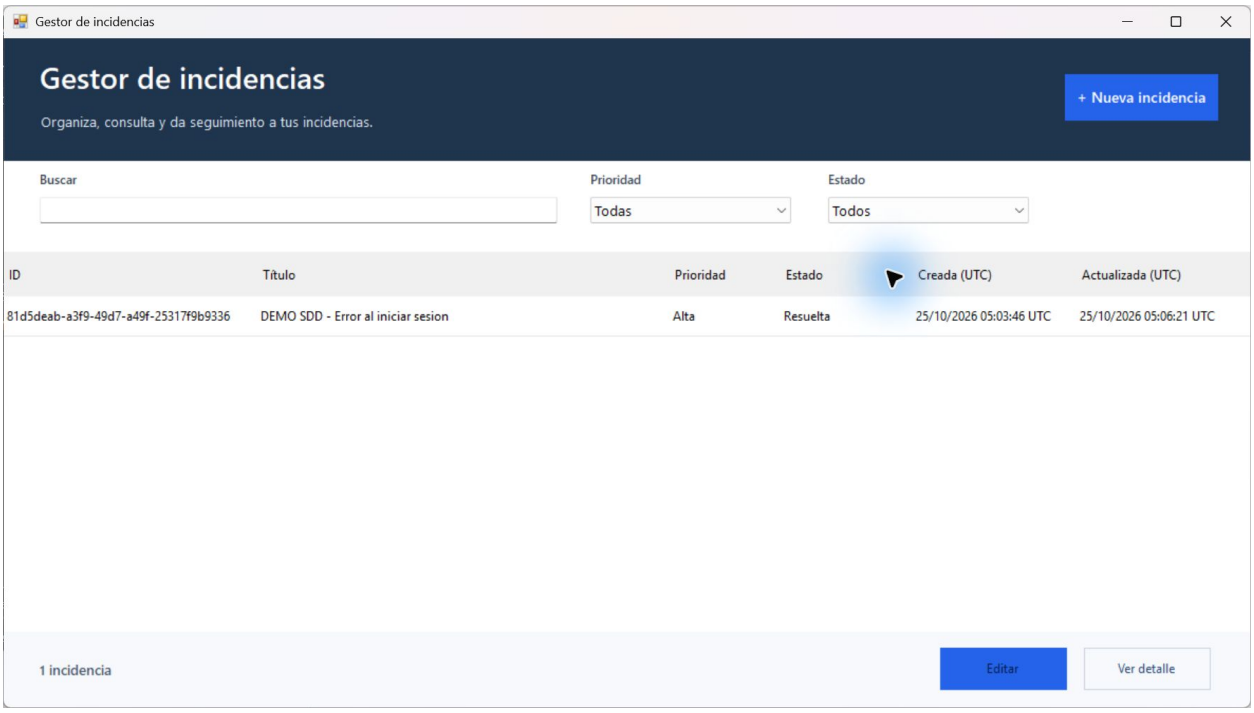
Captura real de la verificación. En el anexo D la comprobación aún estaba en ejecución al capturar.

Anexo B. Detalle resuelto



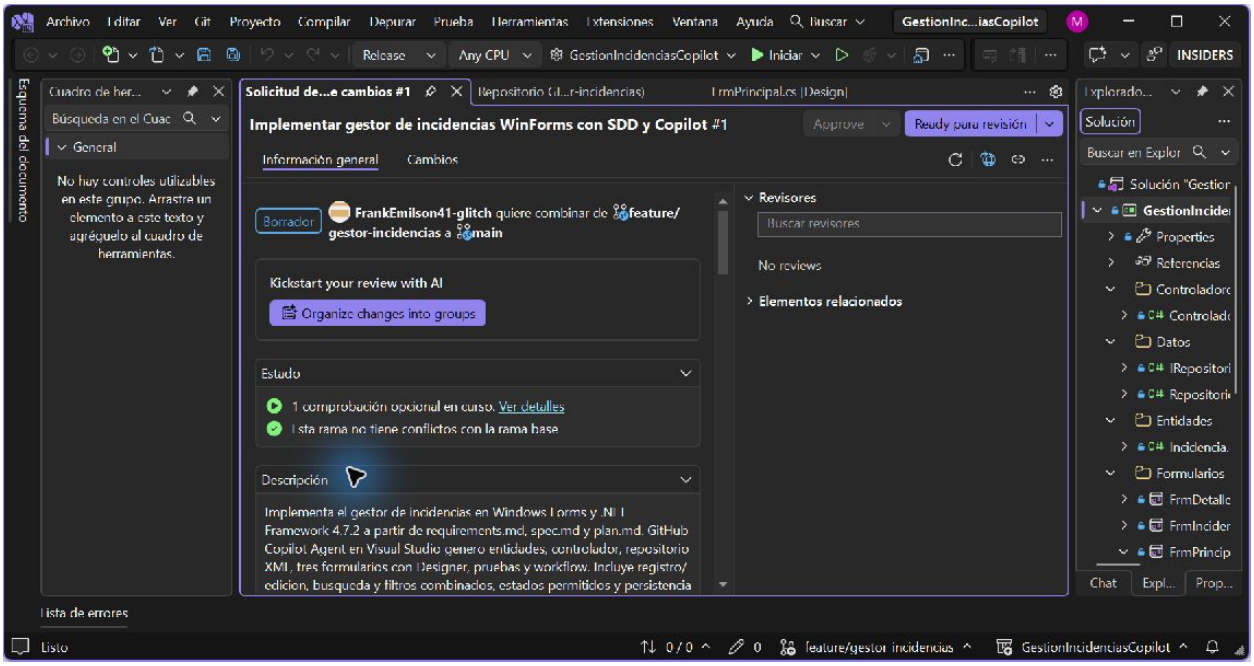
Captura real de la verificación. En el anexo D la comprobación aún estaba en ejecución al capturar.

Anexo C. Persistencia tras reinicio



Captura real de la verificación. En el anexo D la comprobación aún estaba en ejecución al capturar.

Anexo D. Pull request en borrador



Captura real de la verificación. En el anexo D la comprobación aún estaba en ejecución al capturar.